



ข้อสอบปฏิบัติการ (Practical Exam)

วิชา DGT01 1020 Network Programming

อาจารย์ผู้สอน: อาจารย์ภัทรพล ศรีรักษ์

วันที่ 14 พฤษภาคม พ.ศ. 2569

เวลาสอบ: 180 นาที (ข้อสอบถูกออกแบบสำหรับ 120 นาที + 60 นาที สำหรับความล่าช้าในการตรวจ)

คะแนนเต็ม: 100% (ถูก weight เป็น 25% ของคะแนนรวมวิชา)

เนื้อหา: ครอบคลุมปฏิบัติการครั้งที่ 1-4 (HTTP/API, Email/Remote/DNS, Socket/Threading, Selectors/Asyncio)

คำชี้แจง

- ข้อสอบมี 5 ข้อ โดยข้อ 1-4 เป็นข้อย่อย และข้อ 5 เป็น Capstone รวมหลายหัวข้อ
- แต่ละข้อแบ่งคะแนนเป็น checkpoint ย่อย ทำได้บางส่วนได้คะแนนบางส่วน
- ข้อสอบนี้ใช้ localhost (127.0.0.1) เท่านั้น โดยไม่ต้องเชื่อมต่อ digilab.sut.ac.th
- ส่งงานเป็นไฟล์ .py ตามชื่อที่กำหนดในแต่ละข้อ ผ่านระบบ Labpoint
- อนุญาตให้เปิดเอกสาร ไฟล์อิเล็กทรอนิกส์ที่เตรียมมา, Lab 1-4, E-Learning และ Python documentation ได้ระหว่างสอบ ห้ามสื่อสารกัน หรือใช้ AI (Chat GPT, Gemini, Google Search, Chrome Ask Gemini, ฯลฯ)
- ก่อนส่งให้ทดสอบรันโค้ดให้ผ่านเงื่อนไขทุก checkpoint ที่ระบุในแต่ละข้อ

สรุปคะแนน

ข้อ 1 HTTP API + Error Handling (Lab 1)	20%
ข้อ 2 Email Automation: SMTP + IMAP (Lab 2)	20%
ข้อ 3 Multi-client Chat with Lock (Lab 3)	25%
ข้อ 4 Asyncio Echo Server (Lab 4)	10%
ข้อ 5 Capstone: Async Notification System (Lab 2-4)	25%
รวม	100%

ไฟล์ที่ต้องส่ง

- exam_q1.py HTTP API + Error Handling
- exam_q2.py Email automation (Beware: ห้าม hardcode password ใช้ environment variable หรือ input prompt)
- exam_q3.py Threaded Chat Server
- exam_q4.py Asyncio Echo Server
- exam_q5.py Capstone Notification System



ข้อ 1 HTTP API + Error Handling (20%)

ไฟล์ที่ต้องส่ง: exam_q1.py

สถานการณ์

คุณกำลังพัฒนาเครื่องมือสำหรับเก็บข้อมูลสภาพอากาศจาก public API เพื่อใช้ในระบบ monitoring โดยข้อมูลต้องสามารถดึงตามชื่อเมืองที่ผู้ใช้ป้อนเข้ามา และต้องจัดการ error ทุกกรณีที่เกิดขึ้น

API ที่ใช้

Open-Meteo Geocoding API (ฟรี ไม่ต้องใช้ API key):

```
GET https://geocoding-api.open-meteo.com/v1/search
```

Query Parameters ที่ต้องส่ง:

- name = ชื่อเมืองจาก input (เช่น Bangkok, London, Tokyo)
- count = 3 (ดึงผลลัพธ์ 3 อันดับแรก)
- language = en

โจทย์

เขียนโปรแกรมที่ทำตามขั้นตอนดังนี้:

- ☐ รับ input ชื่อเมืองจากผู้ใช้
- ☐ เรียก API ด้วย requests.get() พร้อมส่ง query parameters ผ่าน params= argument
- ☐ ตั้ง timeout = 5 วินาที
- ☐ จัดการ error ด้วย try/except ครอบคลุม HTTPError, Timeout, RequestException
- ☐ ถ้าสำเร็จ แสดงผลลัพธ์ 3 เมืองแรก โดยแต่ละเมืองแสดง: name, country, latitude, longitude
- ☐ ถ้าไม่พบเมือง (response ไม่มี key 'results') ให้แสดง 'Error: city not found'



ตัวอย่างผลลัพธ์ที่คาดหวัง (กรณีสำเร็จ)

```

Enter city: Bangkok
=== Search Results: Bangkok ===
1. Bangkok, Thailand
   Lat: 13.75, Lon: 100.5167
2. Bangkok, United States
   Lat: 39.0334, Lon: -84.4569
3. Bangkok Noi, Thailand
   Lat: 13.7604, Lon: 100.4744

```

ตัวอย่างผลลัพธ์ (กรณีไม่พบ)

```

Enter city: zxcvbnm123
Error: city not found

```

เกณฑ์ให้คะแนน (20 คะแนน)

ใช้ requests.get() พร้อม params= argument ถูกต้อง	4
ใช้ .json() แปลง response	2
ตั้ง timeout=5 และดักจับ Timeout exception	3
ดักจับ HTTPError (เช่น raise_for_status) และ RequestException เป็น fallback	4
ตรวจสอบ key 'results' และแสดง 'Error: city not found' ได้	3
แสดงผลลัพธ์ 3 เมืองแรก format ถูกต้อง (name, country, lat, lon)	4
รวม	20

ข้อ 2 Email Automation: SMTP + IMAP (20%)

ไฟล์ที่ต้องส่ง: exam_q2.py



ห้าม hardcode password ใน source code โดยเด็ดขาด
ให้ใช้ input() หรืออ่านจาก environment variable / .env file
ใช้ Gmail App Password ที่นักศึกษาสร้างไว้ใน Lab 02 ได้

โจทย์

เขียนโปรแกรมที่ทำงาน 2 ส่วนต่อกันในไฟล์เดียว:

ส่วนที่ 1: ส่ง Email พร้อม Attachment

- ☐ สร้างไฟล์ exam_log.txt ในเครื่อง โดยเขียนข้อความ 'Network Programming Exam <ชื่อ-นามสกุล>' ลงไป
- ☐ ส่ง email ไปหาตัวเอง (ใช้ SMTP_SSL ผ่าน smtp.gmail.com:465)
- ☐ Subject: Exam Q2 Report
- ☐ Body: เป็น HTML มี <h2> และ <p> อย่างน้อย
- ☐ แนบไฟล์ exam_log.txt ที่สร้างในขั้นตอนแรก

ส่วนที่ 2: อ่าน Email และยืนยันว่าส่งสำเร็จ

- ☐ ใช้ IMAP4_SSL เชื่อมต่อ imap.gmail.com
- ☐ ค้นหา email ด้วย criterion: SUBJECT "Exam Q2 Report"
- ☐ ดึง email ฉบับล่าสุดที่ตรงกับ subject นี้ออกมา
- ☐ decode subject ที่อาจเป็นภาษาไทยด้วย make_header(decode_header(...))
- ☐ แสดง From, Subject, Date ของ email ฉบับนั้น
- ☐ Mark email นั้นเป็น Seen

ตัวอย่างผลลัพธ์

```
[1/2] Creating exam_log.txt...
[1/2] Sending email with attachment...
[1/2] Email sent!

[2/2] Connecting to IMAP...
[2/2] Found 1 matching email(s)
=== Latest Exam Q2 Report ===
From: piya.ro@g.sut.ac.th
Subject: Exam Q2 Report
Date: Sun, 10 May 2026 14:30:22 +0700
[2/2] Marked as Seen Done
```

เกณฑ์ให้คะแนน (20 คะแนน)

ไม่ hardcode password (ใช้ input หรือ env var)	2
สร้างไฟล์ exam_log.txt และเขียนข้อความถูกต้อง	1
ใช้ MIME multipart + MIME text('html') + MIME base สำหรับ attachment	3
ส่ง email ผ่าน SMTP_SSL สำเร็จ	2
Email ปลายทางได้รับและเปิดไฟล์แนบได้	2
ใช้ IMAP4_SSL เชื่อมต่อและ search ด้วย SUBJECT criterion	3
decode subject ภาษาไทยได้ด้วย make_header	2
แสดง From, Subject, Date ครบถ้วน	3
Mark email เป็น Seen หลังอ่าน	2
รวม	20



ข้อ 3 Multi-client Chat with Lock (25%)

ไฟล์ที่ต้องส่ง: exam_q3.py

โจทย์

เขียน TCP Chat Server แบบ Multi-client โดยใช้ ThreadPoolExecutor + Lock ที่มี requirement ตามนี้:

Server (อยู่ในไฟล์เดียวกัน):

- Bind ที่ 127.0.0.1:9500
- ใช้ SO_REUSEADDR
- ใช้ ThreadPoolExecutor(max_workers=10)

:

- ☐ เมื่อ client connect server ส่งข้อความ 'Enter username: ' กลับไป
- ☐ Client ส่ง username (ขึ้น \n หรือไม่ได้ strip ทั้ง)
- ☐ Server broadcast '*** <username> joined the chat ***' ให้ทุกคน
- ☐ ทุกข้อความที่ client ส่งมา server จะ broadcast ไปยัง client ทุกคน (รวมตัวเอง) ในรูปแบบ '<username>: <message>'
- ☐ ถ้า client พิมพ์ '/who' server ตอบกลับเฉพาะ client นั้น ด้วยรายชื่อ user online ทั้งหมด เช่น 'Online: alice, bob, charlie'
- ☐ ถ้า client พิมพ์ '/quit' หรือ disconnect server broadcast '*** <username> left the chat ***' และลบออกจาก list

Concurrency Requirement (สำคัญ):

- ต้องใช้ threading.Lock() ทุกครั้งที่อ่าน/เขียน dictionary ของ client
- Server ต้องรองรับ client อย่างน้อย 5 คน พร้อมกัน
- ห้ามใช้ selectors หรือ asyncio (ข้อนี้ใช้ thread เท่านั้น)



วิธีทดสอบ

เปิด terminal 5+ หน้าต่าง แล้วใช้ ncat หรือ telnet เชื่อมเข้า server:

```
# Terminal 1 รัน server
python exam_q3.py

# Terminal 2-6 รัน client
ncat 127.0.0.1 9500
# หรือ
telnet 127.0.0.1 9500
```

ตัวอย่างผลลัพธ์

```
[Terminal 2 Alice]
$ ncat 127.0.0.1 9500
Enter username: Alice
*** Alice joined the chat ***
*** Bob joined the chat ***
Bob: hi everyone
Alice: hello bob
/who
Online: Alice, Bob

[Terminal 3 Bob]
$ ncat 127.0.0.1 9500
Enter username: Bob
*** Bob joined the chat ***
Bob: hi everyone
Alice: hello bob
```

เกณฑ์ให้คะแนน (25 คะแนน)

bind 127.0.0.1:9500 + SO_REUSEADDR + listen สำเร็จ	2
ใช้ ThreadPoolExecutor(max_workers=10) อย่างถูกต้อง	3
ขอ username จาก client ก่อนเริ่ม chat	2
broadcast '*** <username> joined ***' เมื่อมีคนเข้า	3
broadcast ข้อความปกติในรูปแบบ '<username>: <msg>' ครบทุกคน	4
รองรับคำสั่ง /who แสดงรายชื่อ online (ตอบเฉพาะคนถาม)	3

รองรับคำสั่ง /quit และ broadcast '*** left ***'	3
ใช้ threading.Lock() ป้องกัน race condition บน clients dict	3
ทดสอบกับ client 5 คนพร้อมกันได้โดยไม่มี exception ที่ unhandled	2
รวม	25



ข้อ 4 Asyncio Echo Server with Counter (10%)

ไฟล์ที่ต้องส่ง: exam_q4.py

โจทย์

เขียน Asyncio TCP Echo Server ที่มีฟีเจอร์เพิ่มเติม:

- ☐ Bind ที่ 127.0.0.1:9600 ด้วย `asyncio.start_server()`
- ☐ รับ client หลายคนพร้อมกันได้ (asyncio handle ให้)
- ☐ เก็บตัวแปร global ชื่อ `message_count` นับจำนวนข้อความรวมที่ server ได้รับจากทุก client
- ☐ ทุกครั้งที่ได้ข้อความจาก client ให้ตอบกลับในรูปแบบ `'#<count>: <original message>'` เช่น `'#1: hello'`
- ☐ ถ้า client ส่งข้อความ 'STATS' (ตัวพิมพ์ใหญ่ทั้งหมด) ให้ตอบเฉพาะ client นั้นว่า `'Total messages: <count>'` และไม่นับข้อความนี้รวมใน count
- ☐ เมื่อ client disconnect ให้ print `'[-] Disconnected: <addr>'` ที่ server console

ข้อกำหนด

- ห้ามใช้ `threading`, `ThreadPoolExecutor`, หรือ `selectors`
- ห้ามใช้ `Lock` เพราะ asyncio coroutine ใน 1 thread ไม่ต้องการ
- ใช้ `async def` และ `await` ให้ถูกต้อง



ตัวอย่างผลลัพธ์

```
# Client A
$ ncat 127.0.0.1 9600
hello
#1: hello
how are you
#2: how are you

# Client B (พร้อมกัน)
$ ncat 127.0.0.1 9600
hi
#3: hi
STATS
Total messages: 3
foo
#4: foo
```

เกณฑ์ให้คะแนน (10 คะแนน)

ใช้ <code>asyncio.start_server()</code> bind 127.0.0.1:9600 สำเร็จ	2
<code>handle_client</code> เป็น <code>async def</code> + ใช้ <code>await reader.read / writer.drain</code> ถูกต้อง	2
echo ในรูปแบบ <code>'#<count>: <msg>'</code> พร้อมเพิ่ม counter ทุกข้อความ	3
รองรับคำสั่ง STATS (ไม่นับรวม count)	2
ปิด writer ด้วย <code>writer.close() + await writer.wait_closed()</code> เมื่อ disconnect	1
รวม	10



ข้อ 5 Capstone: Async Notification System (25%)

ไฟล์ที่ต้องส่ง: exam_q5.py

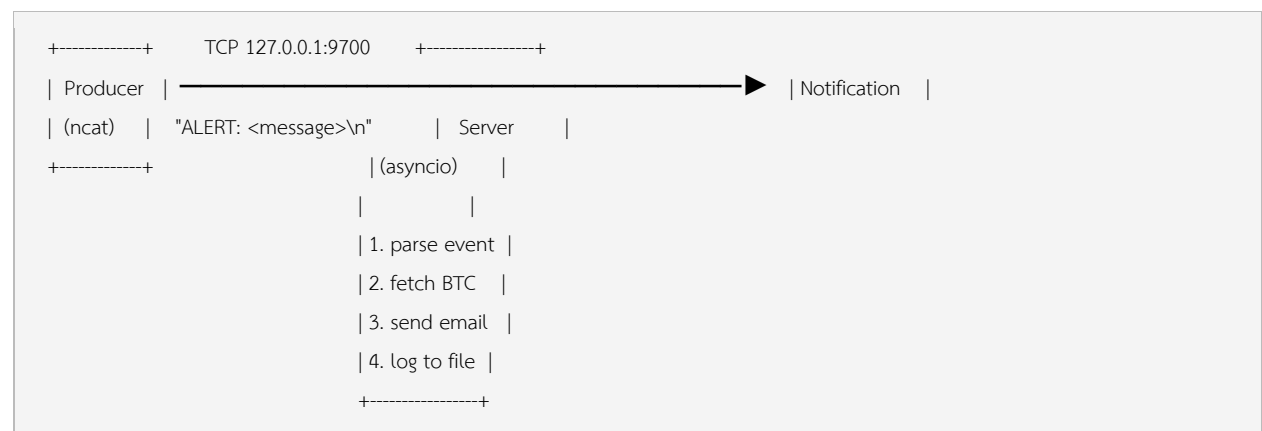
ข้อนี้รวมทักษะจาก Lab 1, 2, และ 4 เป็น scenario จำลองงานจริง

ใช้ localhost เท่านั้น ส่ง email ใช้ Gmail (ของตนเอง) ก็ได้ หรือใช้ smtplib แบบ dry-run โดย print แทนก็ได้ (ดูรายละเอียดที่ Mode A vs B)

Scenario

"คุณเป็น Backend Engineer ของบริษัท Fintech บริษัทต้องการ notification server ที่รับเหตุการณ์ (event) จากระบบอื่นผ่าน TCP socket เมื่อมี event เข้ามา ระบบต้องตรวจสอบราคา BTC ปัจจุบันจาก public API เพิ่มเติม แล้วส่ง email สรุปไปหา admin โดยทุกอย่างต้องไม่ block ใช้ asyncio"

Architecture ที่ต้องสร้าง



ขั้นตอนที่ Server ต้องทำเมื่อรับ event

- ☐ รับ message จาก client ที่ขึ้นต้นด้วย 'ALERT:' (เช่น 'ALERT: high latency detected')
- ☐ ถ้าไม่ขึ้นต้นด้วย 'ALERT:' ให้ตอบกลับ 'INVALID' และไม่ทำอะไรต่อ
- ☐ Fetch ราคา BTC ปัจจุบันจาก:
https://api.coingecko.com/api/v3/simple/price?ids=bitcoin&vs_currencies=usd
- ☐ จัดการ error ของการ fetch (timeout, HTTPError, RequestException) ถ้า error ให้ใช้ราคา 'N/A'

- ☐ ส่ง email สรุป (ดู Mode A/B ด้านล่าง) มี subject 'Alert Notification #<seq>' และ body ระบุ message + BTC price + timestamp
- ☐ เขียน log ลงไฟล์ alerts.log ทุกครั้งในรูปแบบ '<timestamp> | seq=<n> | btc=<price> | msg=<message>'
- ☐ ตอบกลับ client ว่า 'OK <seq>' หลังประมวลผลเสร็จ
- ☐ Server ต้อง handle หลาย client พร้อมกันได้ ใช้ asyncio เท่านั้น

Mode A vs Mode B (เลือก 1 วิธี)

Mode A Real Email (เต็มคะแนน)

ส่ง email จริงผ่าน Gmail SMTP_SSL ไปหาตัวเอง ใช้ App Password ที่สร้างใน Lab 02

Mode B Dry-run (เต็มคะแนนเช่นกัน เลือกถ้าทำ Mode A ไม่ได้)

แทนที่การส่ง email ด้วยการ print ข้อความที่จะส่งไปยัง stdout ในรูปแบบ:

```
[EMAIL DRY-RUN]
To: admin@example.com
Subject: Alert Notification #1
Body:
  Message: high latency detected
  BTC: $67,234.50
  Time: 2026-05-10 14:30:22
```

ทั้งสอง mode ใช้เกณฑ์ให้คะแนนเดียวกัน ระบุที่ต้นไฟล์ว่าใช้ mode ไหนเป็น comment

ข้อกำหนด Concurrency

- Server ต้องเป็น **asyncio** เท่านั้น
- การ fetch HTTP ใช้ requests ได้ แต่ต้อง wrap ด้วย `asyncio.to_thread()` เพื่อไม่ block event loop
- การ send email (ถ้าเลือก Mode A) เช่นกัน wrap ด้วย `asyncio.to_thread()`
- ใช้ counter (sequence number) ที่ไม่ต้องการ Lock เพราะ asyncio รัน 1 thread



ตัวอย่างผลลัพธ์

```
# Server console
$ python exam_q5.py
[NOTIFICATION] Listening on 127.0.0.1:9700
[+] Connected: ('127.0.0.1', 53210)
[1] Processing: high latency detected
[1] BTC: $67,234.50
[1] Email sent (dry-run)
[1] Logged to alerts.log
[-] Disconnected: ('127.0.0.1', 53210)

# Producer
$ ncat 127.0.0.1 9700
ALERT: high latency detected
OK 1
ALERT: database down
OK 2
hello
INVALID

# alerts.log
2026-05-10 14:30:22 | seq=1 | btc=67234.50 | msg=high latency detected
2026-05-10 14:30:31 | seq=2 | btc=67234.50 | msg=database down
```

เกณฑ์ให้คะแนน (25 คะแนน)

asyncio.start_server bind 127.0.0.1:9700 สำเร็จ + handle หลาย client	3
parse 'ALERT:' prefix ถูกต้อง ตอบ INVALID เมื่อไม่ใช่	2
fetch ราคา BTC จาก public API พร้อม timeout	3
จัดการ HTTP error (timeout/HTTPError/RequestException) → ใช้ 'N/A' แทน	3
ส่ง email (Mode A) หรือ print dry-run (Mode B) ครบ to/subject/body	4
wrap blocking call (HTTP fetch หรือ smtp) ด้วย asyncio.to_thread()	3
counter (sequence) เพิ่มถูกต้องและส่ง 'OK <seq>' กลับ client	2
เขียน log ลง alerts.log ในรูปแบบที่กำหนด	2
ห้ามใช้ threading.Lock / ThreadPoolExecutor / selectors	2
โปรแกรมรันได้โดยไม่มี unhandled exception แม้ API ล่ม	1
รวม	25



คำแนะนำในการทำข้อสอบ

- อ่านโจทย์ทั้งหมดก่อนเริ่ม กระจายเวลาตาม weight (Q3, Q5 หนักสุด)
- เริ่มจากข้อที่ตัวเองมั่นใจที่สุดก่อน เพื่อเก็บคะแนนรับประกัน
- Test ทีละ checkpoint อย่ารอจนเขียนเสร็จทั้งโปรแกรมแล้วค่อย test
- ถ้า Q5 ทำไม่ทันส่วน real email ให้ใช้ Mode B (dry-run) ได้คะแนนเท่ากัน
- ถ้าติดปัญหา network/firewall บน Q1 หรือ Q5 ให้แจ้ง TA ทันที

การส่งงาน

ให้ผู้สอนปฏิบัติการในห้องเป็นผู้ตรวจ ตามข้อย่อยที่กำหนด

ขอให้ทุกคนโชคดี

